NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 10.
AI agent for JavaScript debugging: analyzing Console errors, Stack Trace, DOM, and Event Loop

Modern Web Development: HTML, CSS, JS/TS

Deadline: 21.10.2026

1. Aim

Learn to systematically identify the root cause of defects in a JavaScript application using Chrome DevTools: moving from Console error messages and Stack Traces to the actual DOM state and execution order in the Event Loop. Learn to use an AI agent as a debugging partner to formulate and refute hypotheses, rather than as a code-patch generator, and verify every AI recommendation with browser-backed evidence.

Expected outcome: for each detected defect, you can demonstrate the complete chain of **reproduction** → **evidence** → **hypothesis** → **verification** → **minimal patch** → **re-test** and explain why the fix resolves the root cause rather than merely masking the symptom.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- Distinguish between `ReferenceError`, `TypeError`, `SyntaxError`, unhandled Promise rejections (`Uncaught (in promise)`), and logical defects that produce no exceptions.
- Read a Stack Trace, including its asynchronous segments, and differentiate the point where an error **manifests** from the location of its **root cause**.
- Use the Sources panel: breakpoints, *Pause on exceptions*, Scope, Watch, and step-by-step execution.
- Inspect the live DOM via Elements and Console: `$0`, `$()`, `$$()`, `getEventListeners()`, the Event Listeners tab, and DOM breakpoints.
- Explain execution order through Call Stack, task queue, microtask queue, and Event Loop — including the fact that an async function runs synchronously up to the first `await`.

- Formulate prompts for an AI agent providing evidence and constraints, rather than vaguely asking it to "fix this".
- Recognize patches that merely mask symptoms (`try/catch`, `?.`, inflated `setTimeout` delays) and explain why they are unacceptable.
- Structure each fix into an individual minimal commit that can be reproduced and verified independently.

3. Lab tasks

1. Work through the research tasks (§4) and conduct experiments in the browser.
2. Download the provided Courtly Booking starter project, run it locally, and commit the initial state to Git.
3. Locate, investigate, and fix at least one defect in each of the three scenarios: A — Console and Stack Trace, B — DOM, C — Event Loop (§5).
4. Conduct the experiment on event handler execution order (§5.5, C.2).
5. Perform a full regression check across the application (§5.6).
6. Prepare your report (§6) and defend your work orally (§7).

> **Important!** The starter project represents another page of the Courtly service from Lab 7: a venue catalogue with search and booking. It contains deliberately seeded defects across all three scenarios; their exact number and locations are not disclosed. Your personal Lab 7 repository cannot be used for this assignment: JavaScript there was restricted to the mobile menu, whereas here everyone starts from the identical defect baseline to ensure objective evaluation.

4. Research tasks

Before starting the practical part, study the sources and conduct the experiments. Include brief summaries (3–5 sentences per subsection) and experiment results in your report. During defense, you will need to explain these concepts without AI assistance.

4.1. Console and error types

Study:

1. JavaScript error reference — MDN
2. Console overview — Chrome DevTools
3. Console features reference — Chrome DevTools

Investigate: differences between `ReferenceError`, `TypeError`, and `SyntaxError`; why a parse-time `SyntaxError` prevents any line in that script from running, whereas an unhandled `TypeError` only interrupts the current task — leaving the rest of the page

running; what `Uncaught (in promise)` indicates; which defects never log to the Console. How message levels, filters, and **Preserve log** work.

Experiment

In the Console, execute snippets that trigger a `ReferenceError`, a `TypeError`, a parse-time `SyntaxError`, and `Promise.reject(new Error('test'))`. Compare the resulting messages: what is shared, what differs, which one lacks a Stack Trace and why. Then execute `JSON.parse('{')`: this is also a `SyntaxError`, but does it have a Stack Trace — and why does it differ from the earlier case? Explain separately why the **first** Console error is typically more critical than subsequent ones.

4.2. Stack Trace and the Sources panel

Study:

4. Error.prototype.stack — MDN
5. Debug JavaScript — Chrome DevTools
6. Pause your code with breakpoints — Chrome DevTools
7. Debugging asynchronous JavaScript with Chrome DevTools

Investigate: how to read a call stack top-to-bottom; what function names, files, line numbers, and column numbers represent; what the asynchronous call stack (frames marked `(async)`) is and where it comes from. Types of breakpoints: line-of-code, conditional, logpoints, DOM mutation, event listener, and exception (*Pause on uncaught / caught exceptions*). What Scope, Watch, and Call Stack display when execution is paused.

Key question: the topmost stack frame marks where an error **manifested**. Is the **root cause** always located there? Provide an example where it is not.

Experiment

Write a function call chain `a() → b() → c()`, where `c()` accesses a property on `undefined`, and invoke `a()` via `setTimeout`. Find the asynchronous portion in the Console stack trace: which frame marks it, and what does it mean? Then repeat with *Pause on uncaught exceptions* active: what extra information does Sources display compared to the Console alone?

4.3. DOM inspection in DevTools

Study:

8. View and change the DOM — Chrome DevTools
9. Console Utilities API reference — Chrome DevTools
10. Document: querySelector() — MDN
11. NodeList — MDN
12. Event bubbling — MDN

Investigate: how the live DOM in Elements differs from the static HTML file and *View page source* once modified by JavaScript; `$0`, `$()`, `$$()`, `getEventListeners()`, `monitorEvents()`; the **Event Listeners** tab; differences between `querySelector`, `querySelectorAll`, and `getElementById`, and what each returns when nothing matches; live vs. static collections; event delegation. An event listener is bound to a specific DOM node rather than a CSS selector — what are the practical implications?

Experiment

On the starter project page, select any button in Elements and inspect its attached listeners in two ways: via the Event Listeners tab and via `getEventListeners($0)` in the Console. Do the results match?

4.4. Event Loop

Study:

13. JavaScript execution model — MDN
14. Using microtasks in JavaScript with queueMicrotask() — MDN
15. In depth: Microtasks and the JavaScript runtime environment — MDN
16. Tasks, microtasks, queues and schedules — Jake Archibald
17. async function — MDN

Investigate: Call Stack; role of Web APIs (timers, network, user events); task (macrotask) vs. microtask queues; when microtasks run and why their queue is drained completely; where rendering fits into the loop; when a `setTimeout(…, 0)` callback executes and why "0 ms" does not mean "immediately"; when `.then()` callbacks and resumes after `await` execute; why an async function executes synchronously until its first `await`, and what it returns at that exact point.

> The answer "Promises run faster than `setTimeout`" is insufficient. You must explain **which queue** each callback enters, **what** enqueues it, and **when** the Event Loop transfers it to the Call Stack.

Experiment

First write down your prediction, then verify in the Console:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
queueMicrotask(() => console.log('4'));
(async () => {
  console.log('5');
  await null;
  console.log('6');
})();
console.log('7');
```

If your prediction did not match the result, identify where your reasoning diverged.

4.5. AI-assisted debugging

Investigate areas where an AI agent genuinely accelerates debugging: analyzing errors and stack traces, generating competing hypotheses, locating related code, crafting minimal patches, and suggesting verification steps. Conversely, identify where it fails:

- patching symptoms rather than root causes;
- modifying unrelated code;
- hallucinating causes unsupported by the stack trace or codebase;
- proposing `try/catch`, optional chaining (`?.`), or fallback defaults that suppress the error without fixing it;
- suggesting arbitrary `setTimeout` delays instead of rectifying execution order;
- altering intended application behavior instead of restoring it.

5. Practical part: AI-assisted debugging

Courtly Booking is a venue catalogue for the Courtly service: search by city and sport, date selection, venue cards, booking dialog, and bookings list. How the application **must** behave is specified in the brief: any deviation constitutes a defect.

```
        Reproduction
             ↓
    Observations & Evidence
    (Console, Stack Trace, Elements, Sources)
             ↓
     Your Hypotheses
             ↓
    AI Agent: Hypotheses & Verification steps
             ↓
    DevTools Verification
             ↓
      Minimal Patch
             ↓
     Re-reproduction
             ↓
    Regression Verification
```

5.1. Setup

1. Extract the archive into a **separate repository** or a `lab-10/` folder in your shared coursework repository.
2. Make your first commit **unchanged** — this serves as the baseline for all diffs in the report and defense.
3. Serve the project through a local web server (Live Server, `npx serve`, etc.). Do not open via `file://`: the browser blocks ES modules and local `fetch()`.
4. Open the page in Chrome or Edge, DevTools → Console. Turn on **Preserve log**. Clear the Console before each scenario.
5. Do not modify any code until the defect is reliably reproduced and evidence is recorded.

5.2. Rules for working with an AI agent

For each scenario, provide the AI agent with at least: expected vs. actual behavior, reproduction steps, Console messages, Stack Trace (or its relevant portion), code snippet, and for DOM defects — the live element state from Elements.

The initial prompt must **forbid modifying code**. For example:

> Analyze this error as a senior JavaScript debugger. Name 2–3 possible root causes based strictly on the provided Console log, Stack Trace, and code. For each hypothesis, suggest a DevTools check to confirm or refute it. Do not edit any code.

Ask for a minimal patch only after you have **independently** confirmed the root cause in DevTools. If the agent operates inside your IDE, use non-editing mode (ask / plan) for the first step. If the agent has browser access (such as via a DevTools MCP server), that is permitted, but every piece of evidence in your report must be reproduced and captured in your own DevTools.

In your report, state the agent name, version, model, and date of work.

5.3. Scenario A — Console and Stack Trace

Find a defect that causes an unhandled runtime error in the Console during user interaction. If multiple errors occur, choose the one where the point of manifestation and root cause are in **different** locations: steps 4 and 6 are specifically designed for this case. You may fix remaining errors as well — each in its own commit.

1. Reproduce the error and document exact steps.
2. Record the full Console error message and type.
3. Expand the Stack Trace. For each frame, record the function name, file, and line number. Note where the asynchronous stack begins, if present.
4. Pause execution at the point of failure using a breakpoint or *Pause on uncaught exceptions*. Using Scope / Watch, determine which value caused the error and **where it originated**.
5. Formulate at least **two hypotheses** — your own and one from the AI — and verify each. Refuting a hypothesis is valuable: document the evidence that disproved it.
6. Apply a minimal patch at the location of the root cause, rather than where the error manifested.
7. Re-run the scenario. Eliminating the Console error does not necessarily mean the defect is resolved: verify that the application matches the brief.

Mandatory deliverable: the linked progression:

```
Console → Stack Trace → paused scope values → origin of value
        → hypotheses → patch → re-test
```

5.4. Scenario B — DOM

Find a defect where **the Console shows zero errors**, yet the interface behaves contrary to the specification.

1. Describe expected vs. actual behavior.

2. Inspect the element in Elements. Check `id`, `class`, attributes, text content, and the **Event Listeners** tab.
3. In the Console, verify what query methods return (`$()`, `$$()`, `document.querySelector(…)`), and check for attached listeners (`getEventListeners($0)`).
4. Highlight the discrepancy: what the code expects versus what actually exists in the live DOM at execution time.
5. If it is unclear **who modifies the element and when**, set a DOM breakpoint (*Break on → subtree modifications / attribute modifications*).
6. Investigate with the AI agent, apply the fix, and verify.

Explain why live DevTools inspection provides stronger evidence than reading static HTML source code.

5.5. Scenario C — Event Loop

C.1. Defect. Find a defect caused by execution timing: an operation occurs earlier or later than intended by the author.

1. Reproduce the defect. Add temporary `console.log` statements or Sources logpoints at key locations and record the actual sequence.
2. Explain the sequence: what executes synchronously, what enters the task queue, what enters the microtask queue, and when the Event Loop dispatches each callback to the Call Stack.
3. Draw an execution diagram tailored to this specific defect:

```
Call Stack → Web API (timer / network / event) → task queue → Event Loop →
Call Stack
Promise / await ──────────────────────────────→ microtask queue
                    (emptied after each task, before rendering)
```

4. Fix **execution order, not timing**. Solutions such as wrapping code in `setTimeout(…, 500)` are unacceptable, even if the defect appears to vanish. Explain why.

C.2. Experiment: two click listeners. Paste the following into the Console on the starter page:

```
const btn = document.createElement('button');
btn.textContent = 'Event Loop demo';
document.body.prepend(btn);

btn.addEventListener('click', () => {
  console.log('listener 1');
  setTimeout(() => console.log('timeout 1'), 0);
  Promise.resolve().then(() => console.log('promise 1'));
});

btn.addEventListener('click', () => {
  console.log('listener 2');
  queueMicrotask(() => console.log('microtask 2'));
});
```

1. **Before running**, predict the output sequence for two cases: (a) clicking the button with the mouse; (b) running in Console `btn.click(); console.log('after click()');`.
2. Ask the AI agent for its prediction for both cases **without showing** the result.
3. Execute both cases and compare against both predictions.
4. Explain why the sequence differs between (a) and (b). Hint: when exactly is the Call Stack empty?

5.6. Regression verification

After applying all fixes, verify the **entire** expected behavior from the brief (§3), not just the areas you modified: fixing one defect frequently introduces regressions elsewhere. Record results in a table:

Behavior from brief	Before fixes	After fixes
3.1 Header and mobile menu	…	…
3.2 Initial state	…	…
…	…	…

Document any defects you identified but did not resolve, along with your reasoning.

5.7. Commits

Each fix must be an **isolated commit** whose message identifies the scenario and root cause, for example `fix(A): <root cause and what was changed>`. The commit diff represents your minimal patch: during defense it will show whether any redundant changes were made. Temporary `console.log` statements must not be committed.

6. Report requirements

Email subject:

```
[Web] Lab 10: AI agent for JavaScript debugging: analyzing Console errors, Stack Trace, DOM, and Event Loop
```

In the email, include links to your repository and the deployed application — choose either GitHub Pages or Vercel:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

The report must include:

1. research conclusions from §4 and experiment findings;
2. AI agent name, version, model, and date of work;
3. for scenarios **A** and **B**: expected vs. actual behavior, reproduction steps; pre-fix Console logs; Stack Trace (A) or DOM state and Console queries (B); hypotheses and verification methods; prompts and concise AI responses; link to fix commit; post-fix Console logs and behavior;
4. for scenario **C**: observed execution order, explanation, diagram, prompt and concise AI response, link to fix commit; for experiment C.2 — your prediction, AI prediction, and actual results for both cases;
5. regression verification table (§5.6);
6. summary: where the AI genuinely saved debugging time versus where its output required DevTools verification; **one concrete example** of an AI suggestion that would have masked the root cause, with an explanation of what it would have concealed. If the agent located root causes immediately every time, ask it to fix the error exclusively inside the file where it manifested and analyze that suggestion.

Screenshots must provide **actionable evidence** — error messages, stack frames, Scope inspection, attached listeners — rather than merely showing DevTools open.

> **Important!** The written report does not substitute for working code: submission requires the report, the repository with granular fix commits, and the deployed working application.

7. Oral defense

Defense is conducted as an oral interview based on your practical implementation. Even if hypotheses or patches were proposed by an AI agent, you must be able to explain every

modification, demonstrate how it was verified, and answer questions while sharing your screen.

The instructor may roll back your patches (`git checkout` of the initial commit) and request that you reproduce the debugging process live without AI assistance.

Sample questions:

Console and Stack Trace

- Reproduce the defect from Scenario A on the unpatched baseline. Read the Stack Trace without AI help: where did the error manifest, and where is its root cause?
- Pause execution at the failure point and display the problematic value in Scope.
- How does a `TypeError` differ from a `ReferenceError`? What does `Uncaught (in promise)` signify?

DOM

- Demonstrate in DevTools where the discrepancy existed between code and DOM. What listeners are currently attached to this element?
- How do `querySelector`, `querySelectorAll`, and `getElementById` differ? What does each return when no match is found?

Event Loop

- Explain the execution flow in Scenario C using your diagram. Why is adding a `setTimeout` delay not a valid fix?
- Why did execution order differ between mouse click and `btn.click()` in experiment C.2?
- What does an async function return at the moment of its first `await`, and what happens next?

AI

- Which AI recommendation did you reject and why? What would it have hidden?
- How did you confirm that the accepted AI fix addressed the root cause rather than a symptom?

8. Grading

This assignment is worth **3 points** (1 point each for research tasks, practical implementation, and defense), plus **1 bonus point**.

Component	Points	Grading criteria
Research tasks (§4)	1	0.25 points for each subsection §4.1–§4.4: 3–5 sentence conclusion and experiment result with explanation (report item 1)

Component	Points	Grading criteria
Practical part (§5)	1	0.25 points for each scenario A, B, and C: defect reproduced, report contains all required evidence, and an isolated commit resolves the root cause (for C — also experiment C.2). Remaining 0.25 points — regression table (item 5), live deployment, and an analyzed example of an AI suggestion masking root causes (item 6)
Defense (§7)	1	Live debugging without AI: reproducing a scenario defect on the unpatched baseline and identifying its cause in DevTools — up to 0.5 points; two questions from other groups in §7 — 0.25 points each for complete answers
Bonus point	+1	Either submitted before **21.10.2026** OR completed during class

> **Important!** Without oral defense, the research and practical components combined are capped at **1 point**: defense confirms that you understand the fixes implemented, rather than having simply accepted AI output. The bonus point is awarded even without defense.